

Relatório - Atividade Prática 05 - ICG

Nome: José Ricardo Rodrigues de Lucena Filho

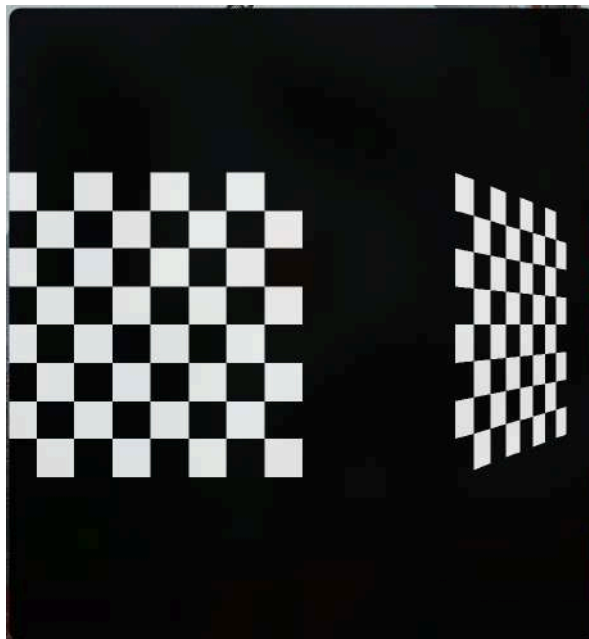
Matrícula: 20210025762

Link: [Atividade no repositório do Github](#)

1. Geração de texturas - checker.c - [link](#)

Seguindo os passos dados, fiz um arquivo para cada mudança pedida no exercício

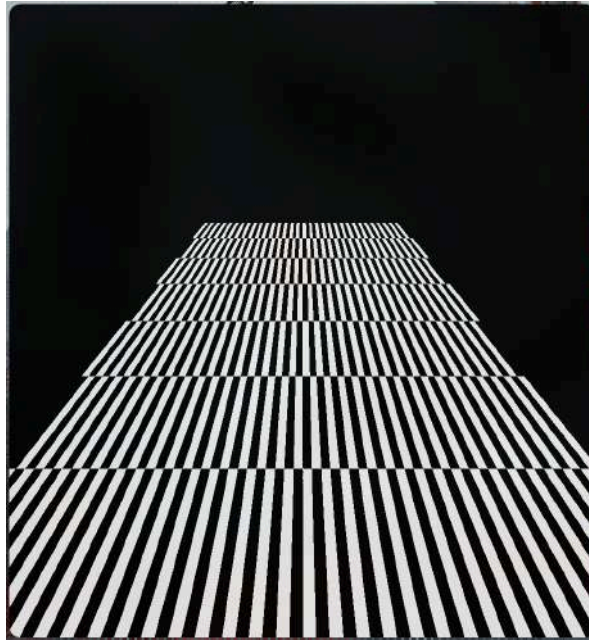
- Executar o código checker.c e observe a imagem renderizada;



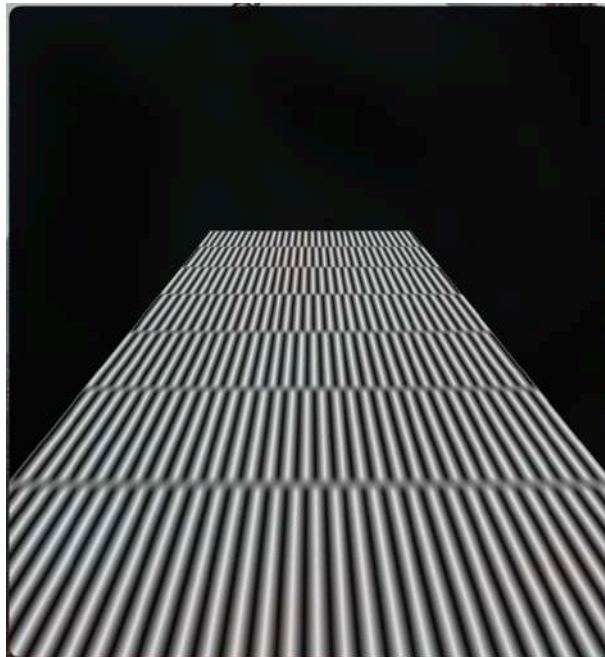
- Remover o segundo quadrado, deixar apenas o da esquerda. Modificar os vértices do quadrado para que fique como a imagem dada;



- Aumentar o número de quadrados na textura, modificando os hexadecimais da linha 64 de 0x8 para 0x1;



- Trocar os parâmetros GL_NEAREST por GL_LINEAR. O que aconteceu?



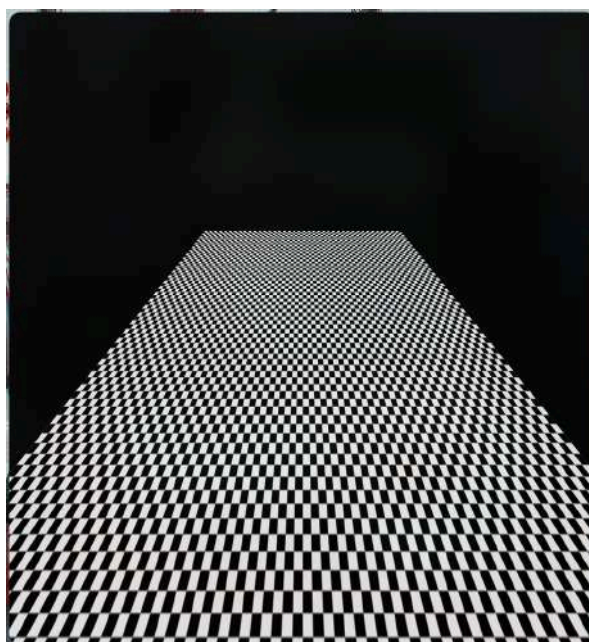
Dá para perceber claramente a diminuição da resolução, visto que essa opção mudou o algoritmo de texture sampling de nearest para bilinear, deixando uma visão mais blurry devido à interpolação entre os 4 vizinhos na imagem da textura.

- Voltar o hexadecimal para 0x8;



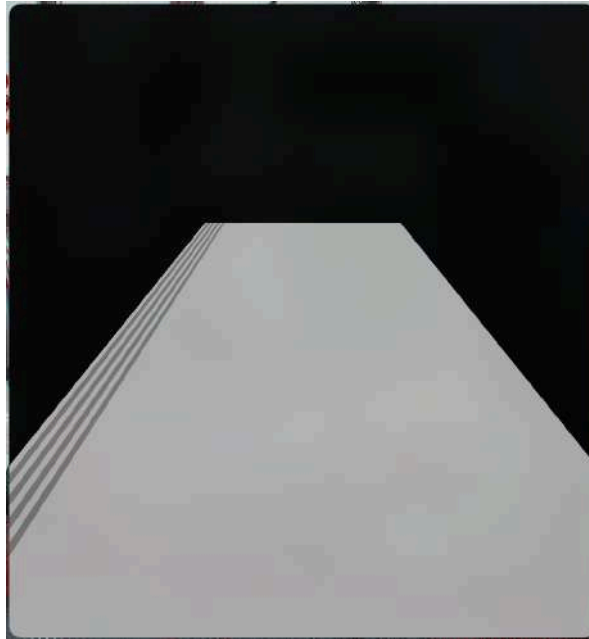
É notável que a texture sampling bilinear ainda deixa as bordas mais borradas/blurry, mas que também isso não afeta tanto nesse caso em que a quantidade de quadrados é menor, pois ficou muito mais aparente no caso anterior, onde tivemos mais quadrados.

- Aumente a escala da imagem de textura mudando seus limites de $(0.0, 0.0) \leftrightarrow (1.0, 1.0)$ para $(0.0, 0.0) \leftrightarrow (10.0, 10.0)$;



Agora aumentamos o número de quadrados. Mas diferente de mudar 0x8 (que muda a densidade do padrão na textura), essa maneira apenas aumenta os limites do tamanho da textura, apenas repetindo o mesmo padrão.

- Mudar o parâmetro GL_REPEAT para GL_CLAMP;



Fica claro que o clamping do GL_CLAMP faz um padrão diferente que o GL_REPEAT, ao invés dos tiles 10x10, ele cria um tile e faz o campling com os texels da borda até cobrir toda a superfície selecionada para a textura.

2. Carregando imagens de textura - checker.c - [link](#)

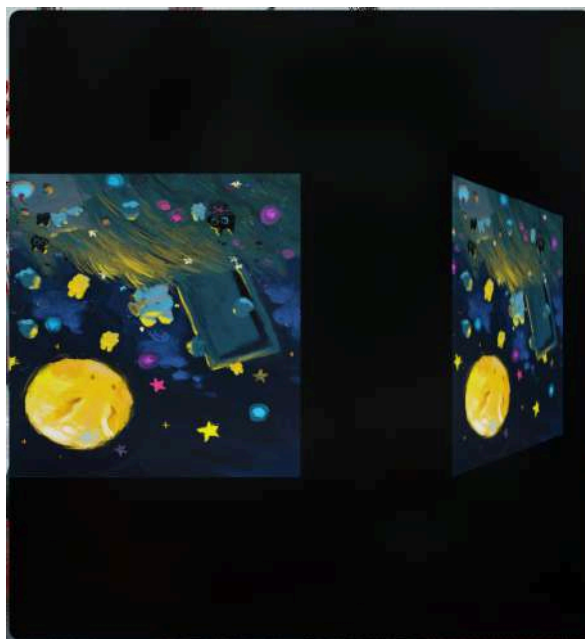
Nessa atividade, foram feitas as mudanças requisitadas, as adições dos métodos extras como o `loadTexture()`, da biblioteca e da pintura em vermelho antes de desenhar a textura, utilizando como textura uma imagem .jpg.

Após as mudanças das bibliotecas e as outras adições, foram feitas várias versões do arquivo, cada uma relacionada à troca do `GL_DECAL` por `GL_REPLACE`, `GL_MODULATE`, `GL_BLEND` e `GL_ADD` para análise dos resultados individuais.

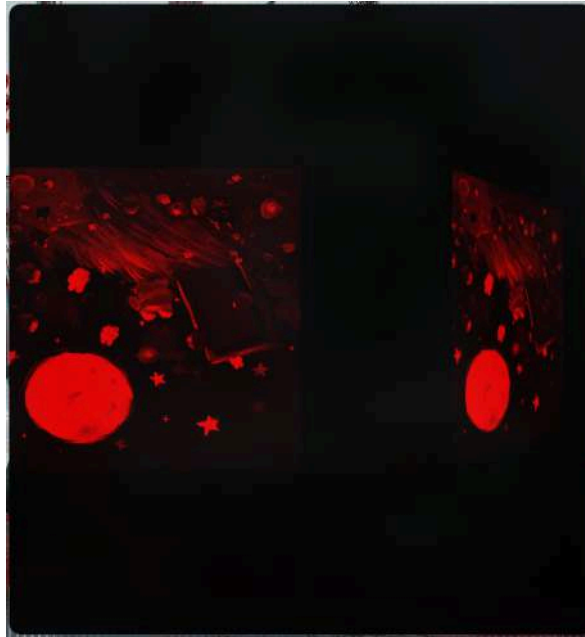
- `GL_DECAL;`



- `GL_REPLACE;`

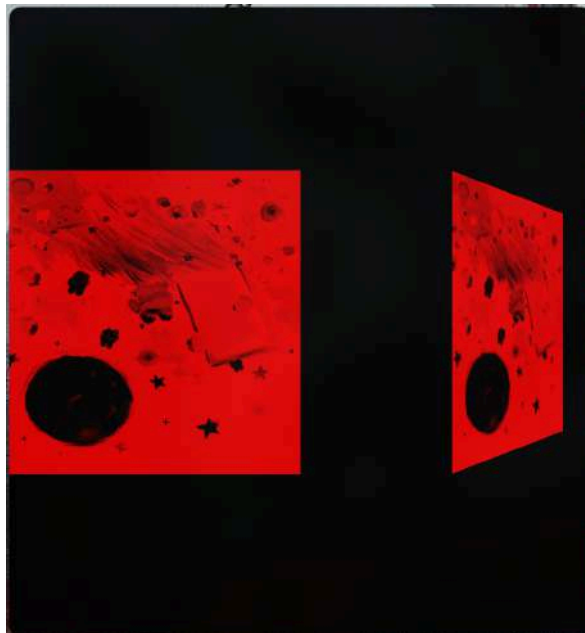


- GL_MODULATE;



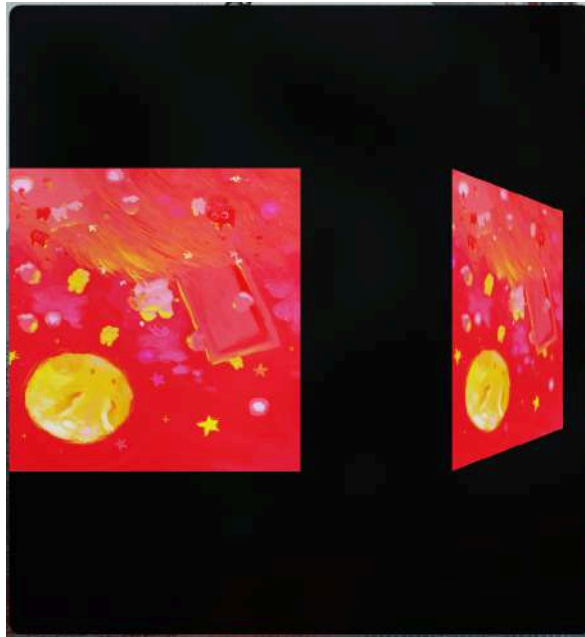
Aqui nota-se que a pintura de vermelho antes de desenhar a textura começou a influenciar a textura mesmo após sobrescrever a pintura em vermelho, mostrando como essa pintura prévia afetou a cor.

- GL_BLEND;



Dá para ver como foi alterado esse efeito vermelho, o blend teve um efeito parecido com uma versão em cor negativa da textura, mas com a cor vermelha que foi pintada antes do desenho dela. Fazendo um blend da textura com a cor.

- GL_ADD;



Essa parece ser a mais básica e direta, fazendo simplesmente uma adição da cor à textura.

Essa análise deixa claro como cada método diferente de texturização afeta a imagem selecionada como textura, e como cores externas à textura podem afetá-la em tempo de renderização.

2.7. Geração de texturas e Carregamento de imagens de textura (Esfera) - checker.c

Agora revisitamos os dois exercícios, mas utilizando os comandos dados para verificar os efeitos agora em uma esfera:

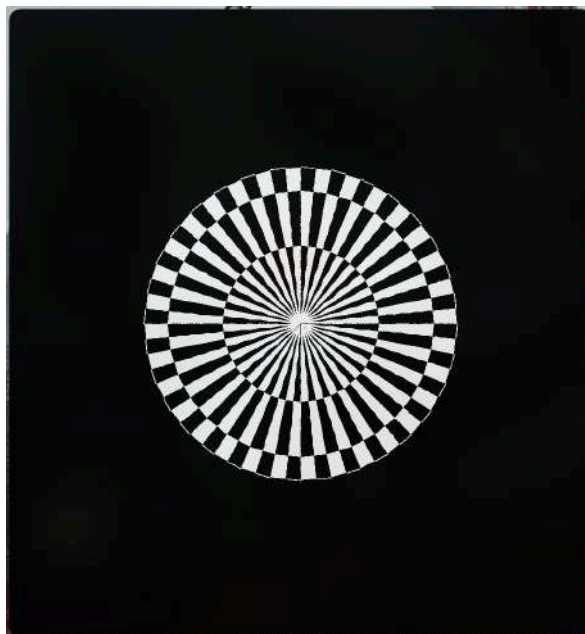
2.7.1. Geração de texturas (Esfera) - checker.c - [link](#)

Feito da mesma forma que anteriormente, cada arquivo diz respeito á uma mudança pedida nas especificações.

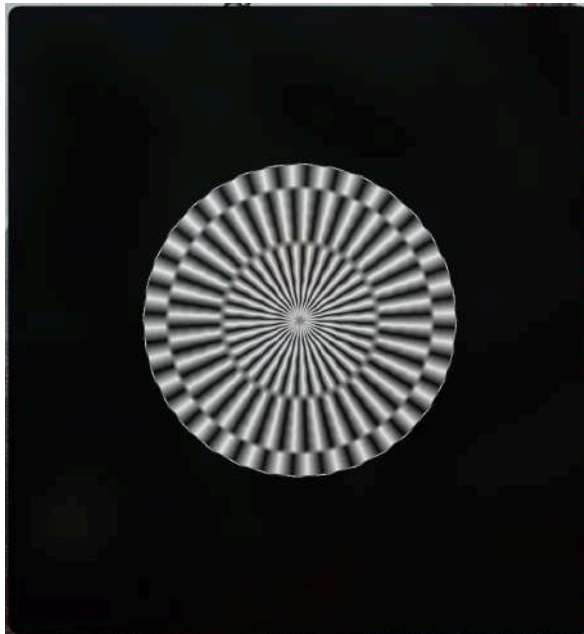
- Executar o código checker.c e observe a imagem renderizada;



- Aumentar o número de quadrados na textura, modificando os hexadecimais da linha 64 de 0x8 para 0x1;



- Trocar os parâmetros GL_NEAREST por GL_LINEAR. O que aconteceu?

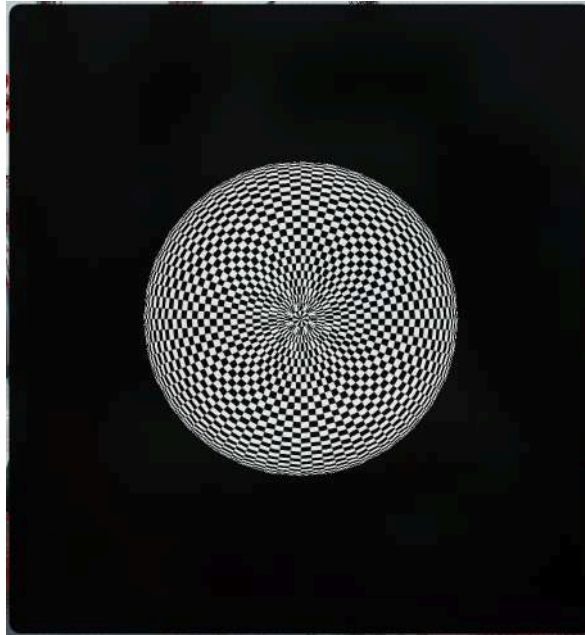


Percebe-se um blurring similar ao visto no quadrado anterior, e pelo mesmo motivo, relacionado ao bilinear filtering.

- Voltar o hexadecimal para 0x8;

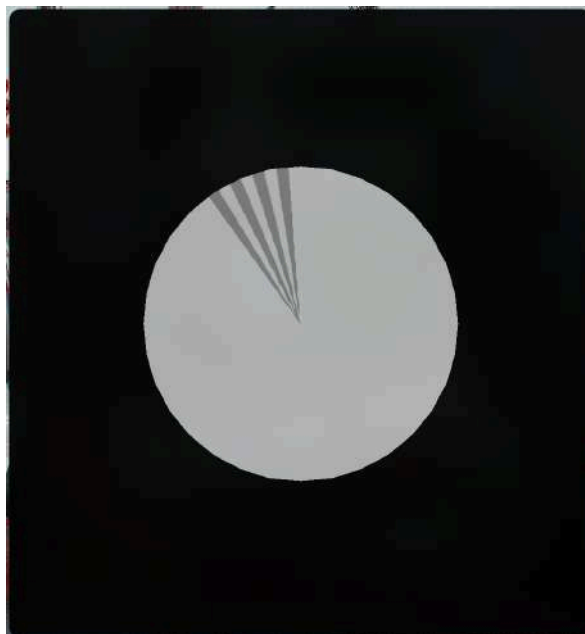


- Aumente a escala da imagem de textura mudando seus limites de $(0.0, 0.0) \leftrightarrow (1.0, 1.0)$ para $(0.0, 0.0) \leftrightarrow (10.0, 10.0)$;



Percebe-se o comportamento equivalente ao visto no quadrado, com o aumento dos limites.

- Mudar o parâmetro `GL_REPEAT` para `GL_CLAMP`;



O mesmo pode-se dizer dessa versão, desenhando apenas um tile e fazendo clamping com os texels da sua borda ao longo da superfície texturizada.

2.7.1. Carregamento de imagens de textura (Esfera) - checker.c - [link](#)

Utilizando a mesma textura, percebe-se resultados similares ao da texturização com quadrado.

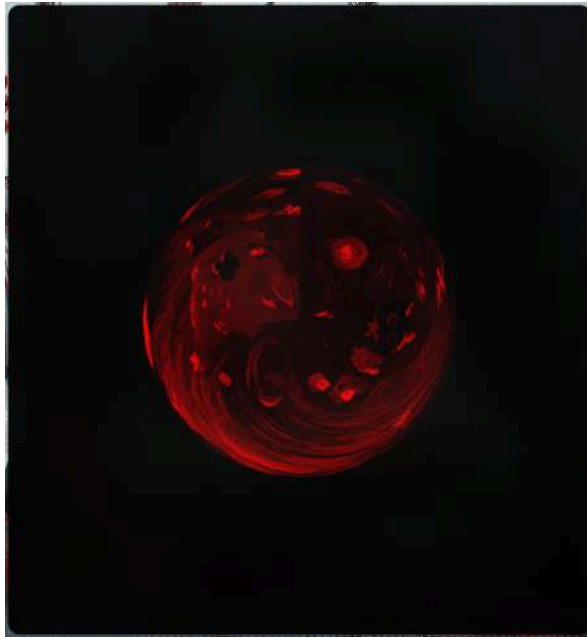
- GL_DECAL;



- GL_REPLACE;



- GL_MODULATE;



- GL_BLEND;



Percebe-se o mesmo efeito de inversão em GL_MODULATE e um efeito similar ao visto anteriormente para GL_BLEND.

- GL_ADD;



Observa-se o mesmo efeito de simples adição no caso do GL_ADD.

Problemas/Dificuldades encontradas

Nesse caso, apenas um problema sobre como inicializar uma variável 'texture' que era chamada na loadtexture(), mas lendo o repositório de onde ela veio foi possível adicioná-la da maneira correta.